AI Loan Adjudication: Final Report

Sabber Ahamed · September 4, 2026

Objective

I treated this project as an MVP for near-real-time, customer-facing loan adjudication. It converts a loan conversation into **Approve, Reject, Human review,** or **Needs information** and explains the outcome using the supplied rules. I also created a Vercel demo UI for an intuitive walkthrough.

System design [more details]

I used a hybrid design that combines an LLM with deterministic rules to improve pipeline reliability and support regulatory requirements. The model remains rule-agnostic, so updated rules can be supplied without additional fine-tuning.

- **Qwen3-1.7B + LoRA** extracts and normalizes ten application fields from the conversation.
- **A deterministic rules engine** makes the final decision and identifies failed rules.
- **Human review** handles missing or conflicting information, invalid output, and model-engine disagreement.

Flow: Dialogue → field extraction → schema validation → rules engine → decision and explanation

Why I used this design

- Lending thresholds and decisions must be repeatable.
- Client rules can change without retraining the model.
- Explanations come from verified rules, making them easier to audit and support.

The fine-tuned model produces a shadow loan decision and a short explanation for evaluation. The rules engine is authoritative: it makes the final decision from the extracted fields and active rules, then compares its result with the model's shadow decision.

Model and training [more details]

Data

I generated synthetic dialogues that collect the ten required fields while varying scenarios such as missing or vague answers, corrections, contradictions, typos, and adversarial prompts.

- **Core split:** 4,000 train, 500 validation, and 500 clean test conversations.
- **Test-2:** 500 adversarial conversations excluded from training.
- **Test-3:** 500 test conversations evaluated under unseen rules without retraining.

The model receives the raw dialogue and predefined rules as input. It returns a flat JSON object containing the extracted fields, a shadow decision, a short explanation, and any failed rule IDs. The deterministic layer then evaluates the extracted fields against the active rules.

Abbreviated training-target example (remaining extracted fields omitted):

```
{
  "employment_status": "unemployed",
  "current_employment_duration_months": 0,
  "decision": "REJECT",
  "failed_rule_ids": ["RULE-EMPLOY-001", "RULE-EMPLOY-002"],
  "explanation": "Employment status and duration do not meet the rules."
}
```

Training setup

- I used one NVIDIA RTX 5090 with 32 GiB of VRAM for dataset-generation experiments and fine-tuning. The final two-epoch, 250-step training run took roughly 45 minutes.
- Qwen's native chat template, with a 2,048-token limit; the longest example was 1,582 tokens
- Dynamic padding and loss only on assistant-response tokens
- QLoRA for two epochs and 250 steps; final adapter size: about 84 MB

I chose Qwen3-1.7B because this is a narrow JSON-output task. The small model and LoRA adapter are easier to serve and version; latency and cost still need production benchmarking.

Artifacts: LoRA adapter · 6,000-record dataset · full metrics and confusion matrices

Evaluation results

Validation was used for model selection. Test-1 measures performance on clean, unseen cases. Test-2 covers adversarial dialogue, including vague answers, accidental statements, prompt injection, and other edge cases. For Test-3, I changed the rules to measure how well the system handles rules it did not see during training. Accuracy measures overall correctness, macro F1 gives each outcome equal weight, and citation exact match requires the complete set of failed rule IDs to be correct.

Overall results

Set	Model accuracy	Model macro F1	Deterministic accuracy	Deterministic citation match
Validation	99.4%	99.3%	99.4%	100.0%
Test-1	99.4%	99.4%	100.0%	100.0%
Test-2	87.8%	87.3%	88.8%	95.6%
Test-3	78.6%	75.1%	91.6%	75.4%

Test-1 model results by outcome

Outcome	Model precision	Model recall	Model F1	Support
Approve	97.7%	100.0%	98.8%	125
Human review	100.0%	99.0%	99.5%	100
Reject	100.0%	98.9%	99.4%	175
Needs information	100.0%	100.0%	100.0%	100

The model made three Test-1 decision errors. Recalculating the decisions with the rules engine produced 100% precision and recall for every class.

Main findings

- **Clean test:** deterministic accuracy and citation match reached 100%.
- **Adversarial test:** the model incorrectly adjudicated 55 of 290 incomplete applications. The rules engine cannot fix a plausible but unsupported extracted value.
- **Changed rules:** deterministic recalculation increased accuracy from 78.6% to 91.6% and reduced false approvals of expected rejections from 19 to zero.

Experiment takeaway

I found that a small model can handle the conversation while deterministic code keeps lending decisions inspectable. The same design supports new client rules without training a separate model for each client.